

The Long And Winding Road — Project Bible

Purpose. This is the single onboarding brief for anyone — human or LLM agent — joining this project. It records what the project *is*, what has been *decided*, what has been *proposed but not signed off*, and what is *still unknown*. Read it in full before making changes.

Revision 3 — 18 August 2026, against branch Issue-7 @ e70b1ac. Rebuilt on both answered question sets.

This document is for both Louis and Damian, and for any agent working on the project. Items needing Damian's specific input are collected in design plan §12 and flagged in §14 here.

0. How to use this document

Every substantive statement carries a status marker. **Respect them.**

Marker	Meaning	What you may do
[DECIDED]	Settled by the project owners, or already true in the repo.	Build on it. Do not relitigate.
[PROPOSED]	Recommended in the design plan, awaiting sign-off.	May be implemented once that phase is approved. Flag that it's provisional.
[OPEN]	Genuinely undecided. Listed in §14.	Do not invent an answer. Ask, or stop and flag it.

If you hit something this document does not cover, **do not fill the gap with a plausible guess**. Add it to §14 and raise it.

The companion **design plan** (docs/design-plan.html) covers visual direction, layout architecture, component specs and the phased build order with the reasoning behind each. This bible is the reference; the design plan is the argument.

1. What the project is

[DECIDED] The Long And Winding Road is a public web toy: **one enormously long road running down the page, drawn collaboratively by anyone who wants to contribute**. Each contributor gets a numbered slot and fills it with their own artwork. The road never ends.

- Slot 1 is at the **bottom**; numbers increase upward. New sections are normally added at the top, but may also fill vacated slots mid-chain (§5.3).
- The road is loaded with an **infinite scroll** system so the page stays performant at any length.
- Every section has a **slot number**, a **title**, a **description**, and an **author name**.
- Contributors **verify an email** but do not create a full account (§6.1).
- It is genuinely unbounded by design — as stated by the owners, *"not us having lofty ambitions, it is showing we can engineer something robust."*

[DECIDED] It is a **joint project** between Louis and Damian. It is a **public toy meant to attract real traffic**, and simultaneously a **portfolio piece for both of them**. That dual purpose is a real design input: engineering that demonstrates capability is valued, and deliberate complexity is acceptable where it earns its keep.

[DECIDED] **Tone**. It does not take itself too seriously, but it is sincere. The humour comes from the premise — a road drawn by hundreds of strangers, presented with total seriousness — not from jokey styling.

[DECIDED] **Visual direction: a modern gallery for art**. Not a gallery of modern art. Generous wall space, restraint, hairlines and alignment instead of cards and shadows, an inventory number on everything.

[DECIDED] **The gallery metaphor governs the space, not the vocabulary**. No museum language in the UI ("plaque", "hang", "accession"), and nothing skeuomorphic — no engraved labels beside each work. The target is *modern web design executed with gallery discipline*.

[DECIDED] **Explicitly not wanted**: the standard "LLM-generated site" look — gradient heroes, glassmorphism, rounded cards with soft shadows on grey, emoji iconography, Inter or Space Grotesk, everything centred. Also not wanted: clinical minimalism. It must have personality.

2. Current state of the repository

2.1 What exists

[DECIDED – factual]

- A Django 6.0 project (aLongRoad/) with two apps: render (the front page) and theme (django-tailwind's generated app).
- One view, `render.views.index_view`, rendering `index.html` at `/`. No context data — page content is hardcoded HTML.
- **No models.** `render/models.py` is empty. Nothing is stored or served from a database.
- **No tests.** `render/tests.py` is the stock stub.
- Two road drawings, `Road_1.png` and `Road_2.png`, hardcoded as example sections. **Both are throwaway test art** and need no migration.
- Eight orphaned images in `render/static/images/` from earlier experiments (`aha.jpg`, `gamer_cat.jpg`, `hippo.png`, `image1-3.png`, `lightning.png`, `pebble.png`) — ~390KB, unreferenced. Safe to delete.
- `test.py` generates `test_for_loading.json`: 250 dummy records under `section_data`, each with `sequential_id`, `author_name`, `description`, `title`. Scratch fixture data.
- `dynamicLoaderDesign.txt` contains two lines of an unfinished sketch. The loader is **not** designed yet (§8).

2.2 Stack

[DECIDED – factual]

Layer	What's actually installed
Backend	Django 6.0, SQLite (dev default, empty)
CSS	Tailwind CSS 4.1 via <code>django-tailwind 4.4</code>
CSS build	PostCSS 8.5 + <code>postcss-cli</code> , with <code>postcss-nested</code> and <code>postcss-simple-vars</code>
Component lib	DaisyUI 5.3 — installed, emitting ~20 colour tokens, used by zero classes
Sass	Not installed. Decision in §9.2.
JS	None, apart from one inline <code><script></code> in the template

2.3 Running it

```
# from the repo root
python -m venv aLongRoadEnv          # name matches the existing .gitignore
./aLongRoadEnv/bin/pip install -r requirements.txt
cd aLongRoad
```

```
../aLongRoadEnv/bin/python manage.py tailwind install    # first time only
../aLongRoadEnv/bin/python manage.py tailwind build      # or `tailwind dev
../aLongRoadEnv/bin/python manage.py runserver
```

Gotcha: Django's AppDirectoriesFinder builds its list of app static directories *once, at process startup*. If theme/static/css/dist/styles.css didn't exist when the dev server booted, building it afterwards won't fix the unstyled page — restart the server. This has bitten this project before.

2.4 Known-broken

See design plan §1 for the full audit. The two that block design work:

- 1. **Mobile shows no content.** Below md, the index column and the entire text panel carry hidden. Phone visitors see drawings and nothing else.
- 2. **Section widths are inconsistent.** Row one splits 25/50/25, row two 20/60/20, and both images are stretched with w-full h-full. Under a chained road this is fatal, not untidy — if the artwork column isn't one exact width for every section, no template can guarantee a join.

3. Vocabulary

[DECIDED] Use these consistently in code, copy, commits and issues.

Term	Meaning	Notes
section	One contributed piece: artwork + title + description + author + slot number.	Not "segment" . Renamed 17 Aug 2026. Some GitHub issue text and the submission flowchart still say "segment" and need updating.
the chain	The full ordered sequence of sections.	Internal name. Chosen because the project intends to accept other forms of transport , not only roads — so "road" is the user-facing word and "chain" is the structural one.
slot	A numbered position in the chain. May be filled, vacant, or reserved.	Slots are allocated, not implicit.
slot number / index	The section's position, displayed as #0001. Slot 1 is oldest, at the bottom.	Padded to 4 digits minimum; numbers keep growing past 9999 rather than changing format.
template	The entry and exit coordinates the server issues when a slot is reserved.	The mechanism that keeps the winding road joined. See §5.

Term	Meaning	Notes
anchors	The four coordinates (entry_l, entry_r, exit_l, exit_r) where the road meets a section's top and bottom edges.	Stored as fractions of artwork width.
head tile	The "add a section" call-to-action at the top of the chain.	
section metadata	The title/author/description/actions block beside a section.	Do not call this a "plaque" or "wall label" in code or copy.

4. Domain model

[OPEN] **No model exists yet** — Issue #3's scope, not yet started. [DECIDED] The frontend is being designed first, so build against fixtures. The fields below are requirements the design places on the eventual model.

Field	Purpose	Status
slot_number	Display index, loader cursor	[DECIDED]
uuid	Public permalink (/section/<uuid>)	[DECIDED] — see §8.1 for the PK caveat
title	Metadata block	[DECIDED] — max ~70 chars [PROPOSED]
description	Metadata block	[DECIDED] — max ~600 chars [PROPOSED]
author_name	Metadata block	[DECIDED] — max ~40 chars [PROPOSED]
entry_l, entry_r, exit_l, exit_r	The chain contract (§5)	[PROPOSED] — fractions of width
image + width + height	Artwork and space reservation	[DECIDED] that both are planned
state	vacant / reserved / pending_review / live / removed	[PROPOSED]
reserved_until	Reservation expiry	[DECIDED] that reservations expire; duration [OPEN]
contributor_email	Verification, notification, rate limiting	[DECIDED]
alt_text		

Field	Purpose	Status
	Optional, moderator- or contributor-supplied	[DECIDED] — never forced
created_at	Ordering, moderation queue	[PROPOSED]

On width/height: the point of storing these is that the browser can reserve the right amount of vertical space *before* the image downloads. Without them the page jumps as each image resolves, which under infinite scroll moves the reader's position under their thumb. This is what the earlier document meant by "height units"; with raster artwork it's simply the image's pixel dimensions.

5. The chain contract

[DECIDED] **Universal anchors were rejected.** The first draft proposed that every section enter and exit at the same fixed coordinates so any piece would join any other. The owners rejected this: *"the key is in the word 'winding'. Yes this makes the project more complicated, but that's sort of the point."*

[DECIDED] The road stays genuinely chained. The join is guaranteed by a **server-issued template**: when a contributor reserves a slot, they are given the exact entry and exit coordinates their artwork must honour.

5.1 The data

Each section carries four numbers describing where the road meets its top and bottom edges — `entry_l`, `entry_r`, `exit_l`, `exit_r` — [PROPOSED] expressed as fractions of the artwork width so they survive any rendered size.

The chain rule:

```
section[n].exit_l == section[n+1].entry_l
section[n].exit_r == section[n+1].entry_r
```

[DECIDED] Road width may **change gradually** within a section, so the two edges are carried independently rather than as a centreline plus a width.

Evidence this is the right shape: measured from the committed PNGs, Road_1 exits at x=213/390 and Road_2 enters at x=214/389 — a hand-authored pair, matching to within a pixel. But Road_2 exits at 312/450 while Road_1 enters at 290/466, a 22px and 16px step. Hand-chaining works only when both pieces are drawn together, which is never true of strangers. Road_2 also narrows to 138 units at its exit against 175–177 elsewhere, which a two-edge template handles and a fixed-width one cannot.

5.2 Slot allocation and the submission flow

[DECIDED] From the submission flowchart (note: it currently uses "segment" throughout and needs renaming):

1. User clicks **add new** on the main page.
2. **Does the user need a custom template?**
3. **No** → assigned the **lowest-numbered available slot**. The slot is reserved for a set period (e.g. 24 hours). User creates or uploads their section.
4. **Yes, with justification** → user sets template parameters themselves, then creates or uploads.
5. **ReviewSection** — a subroutine that reviews the submission and returns a boolean. Details TBD.
6. **Accepted?**
7. Standard path, accepted → **inserted into the live chain**, notification email sent.
8. Custom-template path, accepted → **inserted into an insertion queue** rather than directly live.
9. Rejected (either path) → draft updated with a reason, scheduled for deletion in 30 days, email sent.

[DECIDED] Contributors may **create in an in-browser tool or upload a file** — both are supported, and the template is enforced in **both places**: the frontend for user experience, the backend for security and robustness. Never rely on the frontend alone.

[DECIDED] **Reservation length is not decided** — likely hours to days, and *"won't be longer than a week as that'll leave a patchy road."* It must be **changeable by an admin**, so implement it as a setting rather than a constant.

[DECIDED] **On reservation expiry**: the slot returns to vacant. [OPEN] What happens to the draft — current thinking is that it may persist and be resumable if the slot is still free, and otherwise go into the custom-template queue (§5.4) at lower priority than deliberate submissions.

5.3 Vacant and reserved slots

[DECIDED] A removed section *"would just be treated as an open space and would be assigned to the next contributor to sign up for a space."* Combined with lowest-available-first allocation and time-limited reservations, the chain routinely contains slots with no artwork in them, mid-road, in normal operation.

[DECIDED] **The top of the road is the highest-numbered completed section**, not the highest reserved one. "Newest at the top" is a simplification — because slots are allocated lowest-available-first, a new contribution can land anywhere in the chain. **The road is ordered by slot number, never by submission date.**

[DECIDED] Placeholders for vacant slots are **automatically generated**. A vacant slot cannot be skipped — its neighbours' anchors were issued against it, so removing it breaks the join above and below. It renders the connecting road its template requires, drawn as chrome rather than artwork.

[PROPOSED] Two states, both ordinary list items carrying their slot number so numbering stays continuous, both carrying state as text rather than colour alone:

- **Vacant** — neutral ground, road stroked in - - ink - 3, labelled "This stretch is unbuilt". The whole tile links into the reservation flow.
- **Reserved** — same, road in - - ochre, labelled "Reserved — being drawn". Not a link.

5.4 Custom templates and the routing algorithm

[DECIDED] Contributors who already have artwork — a photo matching a particular format, or a section drawn before signing up — can **set a custom template** specifying the entry and exit points and width they need. These go into a **queue for future addition**, and *"the code that controls the winding of the road will smoothly meet these."*

[DECIDED] The routing behaviour: the algorithm treats queued custom templates as **waypoints**. It *"will aim for those in the queue as waypoints, but won't jump to them. It will wind towards them. It should sort them in the most efficient order to hit them."*

[OPEN] **The algorithm itself is not designed yet, and that is the correct sequencing** — it comes after the main submission flow works.

This is the piece the earlier revisions of these documents missed: **templates are not simply inherited from the section below**. Each time a standard slot is allocated, the algorithm chooses an exit that continues smoothly from the current entry, makes progress toward the next waypoint, and stays inside the canvas.

[PROPOSED] Constraints worth fixing before writing it — design plan §4.3 covers each in more detail:

- **Movement budget**. Define a maximum lateral change per unit of height. Once that exists, reachability is computable rather than guessed, and the whole problem becomes tractable.
- **Width converges too**. Waypoints specify width as well as position, so the steering is two-dimensional. Width should change more slowly than position.
- **Containment**. The road must stay inside the canvas with a margin. Hard constraint, and the most likely source of visible ugliness if added late.

- **It isn't quite the travelling salesman.** Travel is constrained by the movement budget and visiting order is fixed by slot number, so it's closer to trajectory planning with via-points. A greedy "nearest reachable first" is very likely sufficient — worth knowing so you don't reach for a heavier algorithm than the problem needs.
- **Starvation.** Age queue entries so a waypoint with an awkward width doesn't wait forever.
- **Feasibility at submission.** Reject unreachable custom templates immediately with a clear reason rather than queueing something that can never be placed.
- **Issued templates are frozen.** Once handed to a reserved slot, a template cannot change or someone's half-finished drawing stops fitting. Plan forward only.
- **Deliberate imperfection.** Pure shortest-path steering looks mechanical. Add bounded randomness between waypoints — a road that only bends when it has a reason to is the wrong road for this project.

Worth saying plainly: this algorithm is the strongest portfolio artefact in the project. Everything else is careful conventional web engineering; this is a constrained path-planning problem with a real aesthetic objective that arises naturally from the product. When it's built, write it up with diagrams.

6. Contributors, submissions and moderation

6.1 Identity

[DECIDED] **No full account system.** Contributors verify an email address; that is deliberately the whole of it, to reduce friction. An email-verification-only account system (letting people see their past submissions) is a possible later addition, not planned now.

[DECIDED] `author_name` is free text. Impersonation and slurs are handled by moderation rather than by verification.

6.2 Abuse prevention

[DECIDED] Planned: captcha, Cloudflare bot protection, email verification, and a **contribution limit per email** (likely weekly). The stated constraint is not to introduce too much friction.

[PROPOSED] **Add a honeypot**, which was flagged as unfamiliar. It's a form field real people never see and never fill: add an innocuous-looking input (say `website`), hide it in CSS, and set `tabindex="-1"` and `aria-hidden="true"` so

keyboards and screen readers skip it. Naive bots fill every field they find, so any submission with that field non-empty gets discarded. Zero friction for real users, no accessibility cost when hidden properly, and it removes a surprising amount of low-effort spam for free. Pair it with a timestamp check — a form completed in under two seconds wasn't completed by a person. Neither stops a determined attacker (that's what captcha and rate limiting are for); honeypots just clear out the cheap noise. `django-honeypot` implements it.

6.3 Moderation

[DECIDED]

- **Submissions are queued for review before publishing**, not published immediately. The owners note this carries legal weight — see §7.
- Both **automated and human** moderation.
- A **report** control sits next to live sections, asking for **both a category and free text**.
- Reports go to a **report queue** with notification to admins/moderators, likely in **tiers** (admin vs moderator).
- The **Django user system will be used for staff** even though ordinary contributors have no accounts. This is the right call — Django's auth, permissions and admin give you a moderation backend nearly free.
- An **admin/moderator interface is required**.

[OPEN] What happens to a reported section while under review. Current thinking: a weighted threshold — enough reports from users with a low bot-likelihood score triggers automatic withholding pending review.

[DECIDED] **Content rules:** scenery, buildings and similar are fine. Pornography is prohibited. Full guidelines and other red lines are [OPEN].

[DECIDED] Contributors likely **cannot edit** after publishing, and **probably can delete**.

7. Legal and regulatory context

This is not legal advice and no one on this project is a lawyer. It is a researched pointer to the regime that applies, written because two specific worries were raised that turn out to have reasonably clear answers, and because the answers materially change the moderation design. Verify anything load-bearing against current Ofcom guidance, which is still being phased in through 2026.

[DECIDED – external constraint] A UK-operated service letting users upload content other users can see is a **user-to-user service** under the **Online Safety Act 2023**, regulated by **Ofcom**. Small size does not exempt you; duties scale with risk and reach rather than switching off.

7.1 Do you need ID verification? Almost certainly not

The concern was: *"the OSA may have implications on this. Hoping we don't need ID verification."*

The user identity verification duty applies only to Category 1 services — the largest platforms, defined by user-number and functionality thresholds, with Ofcom publishing the register of categorised services. A two-person web toy is nowhere near it.

Even for Category 1 services the duty is to *offer adult users the option* to verify, and the legislation explicitly states the process "need not require documentation to be provided." **The OSA does not create a mandatory real-name or ID policy for general internet use.** Nothing in it requires you to collect ID.

Email verification with no account system remains perfectly viable.

7.2 Age assurance — the question that actually matters

Separate from identity, and where the real obligation could arise.

Services must carry out a **children's access assessment**. If children are likely to access the service, children's safety duties apply. **Highly effective age assurance (HEAA)** — facial age estimation, email-based age estimation, digital ID wallets — is required specifically for services that *allow* pornography and other "primary priority content" (also content promoting suicide, self-harm and eating disorders).

Whether HEAA is required *"depends on whether they prohibit certain types of harmful content and the risk of it appearing on their platform."*

[DECIDED] The project **prohibits pornography**. Combined with pre-publication review, that is the posture that avoids the HEAA requirement — but it makes moderation load-bearing in a legal sense, not just an editorial one. The children's safety duties came into force on 25 July 2025.

What this means practically: do the children's access assessment and write it down. Keep the prohibition explicit in the terms of service. Make sure moderation actually enforces it. That combination is the difference between "no age gate needed" and "you now need facial age estimation", which would be fatal for a casual web toy.

7.3 Does human review make you a "publisher"? No

The worry was: *"If the legal issue is too big, we may be forced to scrap the human review to avoid being publishers."*

This concern is unfounded under UK law, and scrapping review to avoid it would make things worse rather than better.

Defamation Act 2013, section 5 provides a defence for website operators over statements posted by users. The Act specifically addresses moderation: the defence **"is not defeated by reason only of the fact that the operator of the website moderates the statements posted on it by others."** That is the exact scenario worried about, and the statute answers it directly. What *can* defeat the defence is failing to respond properly to a notice of complaint — so having a working report mechanism and acting on it is the thing that protects you, not abstaining from review.

Two supporting points:

- The **hosting defence** (Electronic Commerce (EC Directive) Regulations 2002, reg 19) turns on not having actual knowledge of unlawful content and acting expeditiously once you do. The practical reading is that you must act on what you learn — not that you must avoid learning anything.
- The OSA now **requires** proactive measures against illegal content. A reading of the law where moderating destroys your defences would be incoherent with a regime that mandates moderation.

Recommendation: keep human review. It is the right call for safety, it is what the OSA expects, and the specific legal fear behind dropping it is addressed in statute. If this remains worrying, it is a well-defined question to put to a solicitor — but do not redesign the product around the assumption in the meantime.

7.4 CSAM: what to do on a hit

[DECIDED] Hash-scanning is accepted as sensible. [OPEN] The protocol on a hit — the instinct expressed was to delete immediately, *"but idk if you can report it or should and if so whether deleting it is bad as it's evidence."*

That instinct is half right, and the half that's wrong matters. The distinction is between *making content inaccessible* and *destroying it*. You must do the first immediately. You must not do the second unilaterally.

Best practice, in order:

1. **Quarantine automatically.** On a hash match, make the content inaccessible without a human on the team viewing it. Automated quarantine is not just

convenient — nobody on the project should be opening suspected CSAM to "confirm" it.

2. **Preserve securely.** Retain the file and associated data — account details, timestamps, IP if held — in restricted storage. Guidance is explicit that content should be removed from public access *in consultation with or after reporting to* the authorities, so evidence is not lost. Deleting on sight destroys a chain of custody that law enforcement may need.
3. **Report.** In the UK, report to the **Internet Watch Foundation (IWF)** and/or law enforcement. The IWF provides hash lists, takedown services and a reporting route to members, and is the appropriate first contact for a UK operator.
4. **Ban the account** and preserve its record.
5. **Do not investigate it yourself.** Do not view, download, copy or circulate the material — including to each other for a second opinion. Follow the direction you get from the IWF or police.

[PROPOSED] Write this down as a documented protocol **before launch, not after the first hit**, and make sure both of you know it without needing to look it up. Two people will not want to be improvising at that moment. Note also that hash-matching detects *known* material only — new material still needs the reporting route.

7.5 Data protection

UK GDPR applies to the email addresses you collect, and to IP addresses if you store them for rate limiting. You need a lawful basis, a privacy notice, a retention period and a deletion path. Collecting only an email is a strong position — hold it. Truncate or hash IPs and set a short retention.

7.6 Practical minimum before launch

Terms of service (with the content prohibitions explicit), a privacy notice, a written children's access assessment, a written illegal-content risk assessment, a working report mechanism, a named contact for takedowns, and the documented CSAM protocol from §7.4. Most are short documents rather than projects — but they are genuinely the gate for a UK service accepting public image uploads.

8. The loader

[DECIDED] **The loading mechanics are not designed yet.** An earlier assumption that they were worked out elsewhere was wrong. The owners' position: *"first the basic structure of the page needs to be finished so we know what the JS is working*

with." The page structure therefore defines the contract, and the design plan §8.2 specifies it.

[DECIDED] **Hand-rolled, not the Infinite Scroll library** — it was *"too far away from what we needed"*.

[PROPOSED] The contract the markup must provide:

- **One markup source** — a single `_section.html` partial serves both the server-rendered first batch and every loaded batch. Return **HTML fragments**, not JSON; it keeps markup in one place and removes client-side templating.
- **Cursor pagination on slot number** (`?before=1631`), never offset — offsets shift when a slot is filled or removed mid-scroll and silently duplicate or skip sections.
- **Dimensions known ahead** so sections can be sized before they're fetched.
- **content-visibility: auto** on every section. Measure whether this removes the need for explicit unloading before building unloading.
- **IntersectionObserver on a sentinel**, not a `scroll` listener.
- **If the URL updates on scroll**, use `history.replaceState` throttled — `pushState` stacks an entry per section and destroys the back button. [OPEN] whether to do this at all; flagged for Damian.

8.1 URLs, UUIDs and scraping

[DECIDED] Section permalinks use a **UUID** field on the model (`/section/<uuid>`), chosen so the whole road can't be pulled by walking sequential URLs. Bulk access will be offered **via a separate route** (undecided), with contributors agreeing to a **permissive licence**.

[PROPOSED] **One implementation caveat.** Use a UUID *field* for the public URL, but keep an ordinary integer primary key. Random UUIDv4 as a clustered primary key lands every insert in a random page of the B-tree index — public benchmarks show roughly 3× slower inserts and ~40% larger indexes at scale. If you do want a UUID primary key, **UUIDv7** (RFC 9562, May 2024) embeds a timestamp so values sort chronologically and the fragmentation largely disappears; it's available natively in Python 3.14's `uuid.uuid7()` and PostgreSQL 18's `uuidv7()`. The simplest correct answer for now: `BigAutoField` primary key, indexed `UUIDField` for URLs.

[PROPOSED] **The conclusion that JS pagination is therefore required doesn't follow.** UUIDs stop enumeration of permalinks, which is worth having. But a no-JS "next" link is just an anchor carrying a cursor — `?before=1631` — and following it returns the next batch. That grants a scraper nothing extra, because batch $n+1$ still requires having batch n first: exactly the walk a scroller performs.

Note also that **slot numbers are not secret** — every section displays #1631 on the page. Using the slot number as a cursor leaks nothing that isn't already printed. The UUID protects the permalink; it was never protecting the ordering.

Meanwhile requiring JS buys very little defensively: a headless browser is a few lines of Python and any serious scraper already uses one. The cost falls on screen-reader users, search crawlers and flaky connections, while the determined scraper is unaffected.

What actually reduces load: cache at the edge, rate-limit per IP, and publish the bulk export you already intend to provide, so anyone wanting the dataset takes the cheap path. A documented dump plus a clear licence removes the incentive far more effectively than any technical barrier.

Going JS-only anyway is a defensible call — it's less to build. If you do, keep the **individual section pages server-rendered and crawlable** so sections stay findable and shareable even when the road needs JavaScript. Flagged for Damian in design plan §12.5.

9. Technical decisions

9.1 Tokens and theming

[PROPOSED] Palette, type and scale defined once in Tailwind v4's @theme block — published as both CSS custom properties and Tailwind utilities, one source of truth. Theme via data-theme on <html>, three-state (system / light / dark), default system, with a **synchronous inline** script in <head>. It must be inline and blocking or the page flashes the wrong theme.

9.2 Sass — recommendation: don't add it

[OPEN — recommendation given] Sass was in the brief because it's the CSS tooling Louis already knows, not because anything needs it. That's a reasonable reason to have written it down and deserves a real answer.

The three things people reach for Sass for are now native:

Sass feature Modern equivalent

Variables (\$brand)	CSS custom properties — and unlike Sass variables these are live at runtime , so they can respond to data-theme. Sass variables are compiled away and cannot.
Nesting	Native CSS nesting, supported in every current browser. postcss-nested is already installed as a fallback. @import in the PostCSS chain, which the project already does.

Sass feature Modern equivalent

Partials /
@use

The worry behind the question — *"we may need custom CSS where Tailwind doesn't fully do what we need"* — is correct, and the answer is a `components/` directory of plain `.css` files imported into the bundle. Everything non-utility (the spine grid, the numeral outline, the drawer transition) lives there. Ordinary CSS, nested, using the same tokens, no extra build step.

Worth knowing if Tailwind is unfamiliar: v4 is configured in **CSS**, not in a JavaScript config file. The `@theme` block is the entire configuration surface. Someone who knows CSS well is much closer to knowing Tailwind v4 than they'd have been with v3.

9.3 DaisyUI — Damian's call

[OPEN] Currently installed, emitting ~20 colour tokens, used by zero classes.

The tension: DaisyUI's value is pre-built components with a consistent look, and that look is precisely the generic register \$1 rules out. It also ships a parallel token system (`--color-base-*`, `--color-primary`) that will sit awkwardly beside the `@theme` tokens, and two sources of truth for colour reliably drifts.

Honest middle path if it stays: use it **only** for interactive primitives that are fiddly to build accessibly — the report dialog, the theme switcher — and never for the road, the metadata or the masthead. If it isn't used for that, removing it is free.

9.4 JavaScript approach

[PROPOSED] The interactivity is two things: a disclosure accordion and a scroll-triggered loader. Keep it small.

Option	Cost	What it gives	Verdict
Vanilla ES modules	~150 lines; no build step, no dependency	Full control, transferable knowledge, best fit for a hand-rolled loader	Recommended
htmx (~14KB)	One script tag	Server returns HTML fragments; <code>hx-trigger="revealed"</code> is close to infinite scroll for free. Very Django-idiomatic.	Worth a prototype
Alpine.js (~15KB)	One script tag	Declarative state in markup; tidies the accordion if vanilla gets messy	Reasonable later

Option	Cost	What it gives	Verdict
React / Vue	Build toolchain, hydration, templates duplicated between Django and JS, client routing	Nothing this project needs	Not recommended

On React and Vue specifically, since they were asked about: they exist to keep a complex client-side UI in sync with client-side state. This site's state lives in a database and is rendered by Django, so a framework means maintaining the same section markup twice and fighting for the server-rendered, no-JS, crawlable behaviour you get free otherwise. They're excellent tools aimed at a different shape of problem. Vanilla DOM work here is also more useful to have learned — framework knowledge dates, `IntersectionObserver` doesn't.

9.5 Browser baseline

[DECIDED] Edge, Chrome, Safari, Firefox — current and one previous. Legacy support not a concern.

Internet Explorer needs no consideration at all — Microsoft ended support in June 2022 and it's absent from the modern web platform. Everything recommended (native nesting, content-visibility, :focus-visible, grid-template-rows transitions, AVIF, `IntersectionObserver`) works across that baseline. The only prefixed item is `-webkit-line-clamp`, which is universally supported but still needs the prefix.

9.6 File structure

[PROPOSED] See design plan §9.6. Key point: namespace templates and static under `render/` — the current flat layout lets any app shadow `index.html`, and namespacing is the standard Django fix.

10. Images

[DECIDED] **Sections are the contributor's canvas**. They may fill the whole rectangle, not just draw a road line. The site **must not** filter, invert, dim or recolour artwork in dark mode — *"to violate this in the name of accessibility would be to build a different site."* Dark mode applies to the page around the artwork only.

[PROPOSED] **JPEG is the wrong default**, despite being the current assumption. It has no transparency and its block-based compression produces visible ringing

around exactly the hard edges and flat colour that hand-drawn sections are made of.

Since every upload must be processed server-side anyway (dimensions validated, anchors checked, aspect ratio constrained), transcoding is nearly free to add:

- **Accept** PNG, JPEG, WebP. Keep the contributor's original untouched.
- **Serve** AVIF with WebP fallback via `<picture>`. Both are supported across all current major browsers; AVIF is dramatically better than JPEG on flat-colour illustration.
- **Generate** two or three widths with `srcset/sizes`, since the rendered artwork width varies from ~280px to ~544px.
- **Strip EXIF** on re-encode. Phone photos carry GPS coordinates; a project that wants minimal data shouldn't republish a contributor's location.
- **Guard the decoder.** Set `Image.MAX_IMAGE_PIXELS`, cap upload size, validate dimensions before processing — decompression bombs are the standard attack on any service accepting images.

Pillow is sufficient at this scale and is already a Django dependency for `ImageField`. `pyvips` is much faster if throughput ever matters, but starting there is premature.

10.1 On people disliking WebP

[PROPOSED] The complaint is real but it's about **saving**, not viewing: someone right-clicks, gets a `.webp`, and finds their older software won't open it. Nobody objects to WebP they never have to handle.

That maps onto a clean fix. Serve AVIF/WebP for **display**, where the bandwidth saving is the point, and give each section an explicit **Download** action handing over the contributor's original file in the format they uploaded. Better in both directions — visitors get fast pages, and anyone who actually wants the artwork gets a real PNG or JPEG instead of extracting one from a page. It also fits the permissive-licence intent: if the work is meant to be reusable, make it properly downloadable.

10.2 Security of the upload path

Uploads are the riskiest surface on the site. Design plan §11.2 covers this in full; the essentials: never trust the declared file extension or content type, set `Image.MAX_IMAGE_PIXELS` and a size cap against decompression bombs, randomise stored filenames, serve uploads with `X-Content-Type-Options: nosniff` and ideally from a separate origin, and **never store user-supplied SVG** — SVG is XML that can carry `<script>`, making it stored XSS if served from your own domain. If the in-browser tool emits SVG, rasterise it server-side.

11. Accessibility — non-negotiables

[DECIDED] **WCAG 2.2 AA, self-imposed** — *"partly because it's good for portfolio, but mainly because it's the right thing to do."* Verified, not assumed.

Hard requirements. Do not ship changes that violate them.

- **Content parity across viewports.** No content may be hidden at any breakpoint. The prototype violates this and it's the most serious defect present.
- Exactly one `<h1>`. Section titles are `<h2>`. No level skipped.
- The road is `<ol reversed>`; each section `<li value="N">` containing `<figure>` and `<figcaption>`. The visual numeral is aria-hidden because `<li value>` already conveys position.
- One `<header>`, one `<main>`, one `<footer>`. Skip link first in tab order.
- 4.5:1 contrast for body text, 3:1 for large text and UI boundaries — **in both themes, checked.**
- `:focus-visible` ring contrast-checked against both grounds. Targets $\geq 44 \times 44$ CSS px.
- Reflow to 320px and zoom to 400% with no horizontal scrolling.
- **No hover-only affordances.** Actions are always visible.
- No information by colour alone.
- Full prefers-reduced-motion path.

Unbounded-list specifics: aria-busy while loading; a polite live region announcing loaded ranges (not per scroll event); a **"jump to the end of the road"** control, because the bottom of an infinite list is otherwise unreachable by keyboard; and correspondingly **nothing essential below the road** — no footer navigation, no legal links. Those go in the masthead or on their own pages.

11.1 Alt text

[DECIDED] Alt text is **not forced** on contributors. A moderator may add it, and it might be auto-generated. Nothing is settled beyond "don't force it".

[PROPOSED] The recommendation, stated plainly:

Every `<img>` needs an `alt` attribute. It can either describe the picture, or be **empty** (`alt=""`), which tells a screen reader "skip this, it isn't the content". Empty is *not* the same as missing — a missing `alt` makes screen readers read out the filename, the worst outcome.

Use `alt=""` on the artwork, with the caption carrying the meaning. The author has already written a title and description of their own drawing, and that

text sits in the `<figcaption>` immediately after the image, reached as ordinary text. Duplicating it into `alt` makes screen reader users hear it twice.

If a moderator adds a description later, put it in `alt` then. **Do not auto-generate `alt` by concatenating title and description** — that's exactly the double-announcement problem. Never ship a placeholder like `alt="image1"`.

On contrast and user artwork: WCAG contrast requirements apply to text and interface, not to artwork. Since the site never overlays text on a section image, the artwork is out of scope — which is the technical reason "don't restyle the canvas" costs nothing in compliance terms.

11.2 Testing

[DECIDED] No screen reader currently available; willing to install free ones.

Free options: **Orca** (preinstalled on most Linux desktops — available immediately on the current dev machine), **NVDA** (Windows, free and open source), **VoiceOver** (built into macOS and iOS), **TalkBack** (built into Android). Between a Linux machine and a phone you can cover three of the four at no cost.

12. Operations

12.1 Hosting

[OPEN] Not decided. Cloudflare is of interest; DigitalOcean has been used before.

On Cloudflare Workers and Django specifically, since it was raised: it is now *technically* possible via community packages (`django-cf`, `django-on-workers`) running under Python Workers/Pyodide, but it requires a paid Workers plan, carries cold starts reported anywhere from 300ms to several seconds for Django, and constrains you to what Pyodide supports. It is not a comfortable fit for a Django app doing image processing and database work.

[PROPOSED] The arrangement that gets the Cloudflare benefits without the Workers constraints:

- **Django on a VPS or PaaS** — DigitalOcean (already familiar), Hetzner (cheapest), or a PaaS like Railway, Render or Fly.io if you'd rather not administer a server.
- **Cloudflare in front** as CDN, cache and bot protection — which is also where the bot protection in §6.2 comes from.
- **Cloudflare R2 for artwork** rather than the app server's disk. Zero egress fees, S3-compatible, works with `django-storages`. Object storage is the right home for user uploads regardless of host.

12.2 Database

[OPEN] Not decided.

[PROPOSED] **SQLite for development, PostgreSQL for production.** Reasons: real concurrent writes (SQLite serialises them, which matters once a moderation queue and a public site are writing at once), proper full-text search when section search arrives, and managed Postgres offered by every PaaS. Django makes the switch trivial provided you avoid database-specific features early — so just don't reach for SQLite-specific tricks.

12.3 CI

[DECIDED] No pipeline yet; interested in setting one up.

[PROPOSED] **GitHub Actions**, since the repo is already on GitHub. A useful first pipeline is small: run `manage.py check`, run the test suite, build the Tailwind bundle to prove it compiles, and — once Phase 7 lands — run `axe-core` against a rendered page. Start with the first three; they take about twenty lines of YAML and catch most of what breaks.

12.4 Analytics and error tracking

[DECIDED] Analytics wanted, but minimal data — *"I don't want to be tracking people for a web toy... I just don't want people's data."* Specifically wanted: **page views, per-section view counts, and rough location**. Ideally no cookie banner. Social sharing links wanted, but without tracking.

Those requirements split into three problems with three different answers, and none of them requires tracking individuals.

Per-section view counts — build it yourself, no analytics tool involved. This is your own data about your own content, not third-party analytics. Increment a counter server-side when a section is rendered. Use an `F()` expression so it's atomic and race-free. At scale, batch increments through the cache and flush periodically rather than writing on every view. **No cookies, no JavaScript, no personal data** — you're counting events, not people.

Rough location — take it from Cloudflare, not a script. If you're behind Cloudflare you get a `CF-IPCountry` header on every request, free. Store **aggregate counts per country**, never per-visitor rows. Country-level counts against a section are not personal data; a log of which IP viewed what is. The distinction is entirely in what you choose to store.

Page-level analytics — pick a cookieless tool:

Tool	Model	Notes
Cloudflare Web Analytics	Free	No cookies, no personal data. Obvious fit if already behind Cloudflare.
Plausible	Paid hosted, or self-host free	Open source, EU-hosted, very light script.
Umami	Self-host free	Open source, similar model.

On the cookie banner: if you set no non-essential cookies and use genuinely cookieless analytics that don't store or retrieve information on the visitor's device, the UK PECR consent requirement generally isn't triggered — so no banner. That is a real reward for the restraint, and it is easy to lose by adding one careless script later.

Social sharing without tracking: use plain links (<https://bsky.app/intent/compose?text=...>, <https://twitter.com/intent/tweet?url=...>), **never the official share widgets or SDKs**. Those buttons are tracking scripts that report every page view to the platform whether or not anyone clicks. A plain `<a>` sends nothing until someone chooses to share, costs no JavaScript, and looks however you want.

Errors: **Sentry** has a free tier and can be self-hosted. Configure it to scrub PII — it captures request data by default, which will include more than you want.

13. Conventions and process

13.1 Team

[DECIDED]

- **Damian** — second-year (going into third-year) Computer Science undergraduate in the UK. More frontend-leaning. Knows Tailwind. Owns the DaisyUI decision.
- **Louis** — starting Computer Science at a UK university next September, or a related degree apprenticeship. A* in A Level Computer Science, substantial programming experience. More backend-leaning, knows Sass, learning Tailwind. Wants to get better at frontend.
- Neither is rigid about the split; both want to do some of both.

13.2 Branches and issues

[DECIDED]

- Repo MBDev - LS/aLongRoad, default branch `main`, SSH remote.

- **Issue #1** — "Basic front page with infinite scroll" — the front-end epic. **Issue #5** (Tailwind, closed) and **Issue #7** (dynamic loading tool) are *sub-issues* of #1.
- **Issue #3** — backend road rendering logic. Separate epic, not started.
- **Branch strategy**: front-page and template changes go on Issue - 1 and merge forward into Issue - 7. Loader scratch files go directly on Issue - 7.
- Commit messages referencing an issue are prefixed #N.
- Both contributors work on `index.html` — coordinate before rewriting shared files.

13.3 Turning the plan into issues

[DECIDED] The design plan's phases should **not** be auto-converted into GitHub issues. Guidance requested instead.

How to break the phases down. Each phase in the design plan is roughly an epic, not an issue. A good issue here is one that can be finished and reviewed in a sitting and has a checkable end state. Phase 1 ("The spine"), for example, breaks into: *replace the 100-column grid with the three-track spine, add the four anchor fields to the fixture data, reserve section height from image dimensions, and remove the positional class names*. Each has a clear done condition; the phase's exit criteria become the acceptance criteria for the last one.

Use the **sub-issue** feature you already use for #5 and #7: make the phase a parent issue and the tasks sub-issues. GitHub shows a progress bar on the parent, which makes the phase's state readable at a glance.

GitHub Projects (the Trello-like boards). Projects is a separate layer that sits over issues rather than inside a single repository. Practical setup for two people:

1. Create a Project from the repo's **Projects** tab, using the **Board** template.
2. Add a **Status** field with columns like Backlog → Ready → In progress → In review → Done. Keep it to five at most; more columns than people is friction.
3. Add custom fields that answer the questions you'll actually ask: **Phase** (single-select, 0–7) and **Owner**. Skip estimates — they're noise on a two-person project.
4. Turn on the built-in **workflows** so items auto-move: newly-added items to Backlog, reopened to In progress, closed to Done, merged PRs to Done. This is the single highest-value thing to configure, because a board nobody updates by hand is worse than no board.
5. Use the **Table** view for planning and grooming and the **Board** view for daily work — they're views of the same data, not separate lists.
6. Link PRs to issues with Closes #12 in the PR description so the board moves itself when work merges.

The trap to avoid is treating the board as the source of truth. The issues are the source of truth; the board is a view over them. Never put detail in a Project item that isn't in the issue.

13.4 Code conventions

[PROPOSED]

- Design tokens live in Tailwind v4's @theme block — one source of truth, published as both CSS custom properties and utilities.
- No inline `<script>` in content blocks — JS goes in a static ES module loaded from the `extra_js` template block.
- Class names describe **role**, not position. The prototype's first-child ... fourth-child are exactly what not to do.
- Templates and static namespaced under `render/`.

13.5 Where these documents live

[DECIDED] Both documents live in `docs/`, and **`docs/` is gitignored for now** — they are not to be pushed to GitHub yet.

Consequence: Damian will not receive these through the repository. Share them directly, or remove the ignore rule when you're ready.

14. Open questions

Most of the first two rounds are now answered. What remains, grouped by who needs to act.

A. Needs Damian (design plan §12 sets each out in full, readable standalone)

1. **How wide is the artwork column?** One number serves a 320px phone and a 2560px monitor. Anchors are fractions so any width works mechanically — what's at stake is legibility and consistency. The binding constraint is that desktop flanks need ~36ch for metadata, capping - - road -w around 45% of viewport. Also needs a published **canonical authoring width** (1600px suggested) so contributors know what they're drawing for.
2. **What are the height limits?** Expressed as a ratio to width. Maximum is an experience question — past ~1.5x width a section stops reading as part of a journey. **Minimum is set by the metadata block, not aesthetics:** title, author, description and actions need ~180-220px beside the artwork, so below ~0.4x width the metadata is taller than the art it describes. Suggested starting range: 0.4x to 1.5x.

3. **DaisyUI — keep or remove?** Held open. Section 9.3 states both cases.
4. **Should the URL update as you scroll?** If yes, `replaceState` throttled, never `pushState`. Also worth deciding whether this or the per-section permalink is the primary sharing mechanism.
5. **No-JS fallback — build it or not?** Section 8.1 argues UUIDs don't force JS-only. Skipping it is defensible; if you do, keep individual section pages server-rendered.

B. Needs both of you

1. **Where is artwork stored?** R2 recommended (section 12.1). To discuss together.
2. **Hosting and database.** Recommendations in sections 12.1 and 12.2 — Django on a VPS/PaaS with Cloudflare in front, Postgres in production. To discuss together.
3. **Report destination.** The queue and notification model is decided; the delivery mechanism isn't. Options: Django admin queue only (simplest, no extra infrastructure); admin queue plus email to moderators (better response time); admin queue plus a Discord or Slack webhook (fastest, and free — a webhook POST is about five lines). Recommendation: start with the admin queue plus a webhook, since it needs no mail deliverability work.
4. **Reservation length.** Undecided, hours to days, under a week. Must be admin-changeable, so ship it as a setting and defer the number.

C. Deferred by design, listed so they aren't forgotten

1. **The routing algorithm** (section 5.4). Correctly deferred until the submission flow works. The constraints listed there are worth fixing before writing it.
2. **Content guidelines and red lines.** Acknowledged as complicated and deferred — but note section 7.2: the pornography prohibition is load-bearing for avoiding age-assurance requirements, so that specific line needs to be explicit in the terms of service before launch even if the rest waits.
3. **Draft handling on reservation expiry** (section 5.2). Tentative thinking recorded; not settled.
4. **Custom-template queue insertion policy** (section 5.4). Depends on the routing algorithm.
5. **Field length maximums.** Proposed at 70/40/600 characters; confirm once the layout exists.
6. **Whether an email-verification-only account system** (letting contributors see past submissions) gets built later.

D. Before launch, not yet started

1. **The written documents** in section 7.6 — terms of service, privacy notice, children's access assessment, illegal-content risk assessment, CSAM protocol. Short documents, but genuinely the gate for a UK service accepting public image uploads.
2. **SECRET_KEY rotation.** It is in git history, so it is compromised regardless of what happens next. Rotate before any deploy.

15. Do not do these

[DECIDED] Standing constraints for anyone picking up the project.

1. **Do not use "segment".** The term is "section" (§3). The submission flowchart still needs renaming.
2. **Do not restyle, filter, invert or dim section artwork** for dark mode or any other reason (§10).
3. **Do not hide content at any breakpoint** to solve a layout problem (§11).
4. **Do not add a gap, border, margin or border-radius between sections** — it breaks the chain (design plan §3.3).
5. **Do not skip vacant slots when rendering** — the neighbours' anchors depend on them (§5.3).
6. **Do not reintroduce JS-driven text clamping.** Use CSS `-webkit-line-clamp`.
7. **Do not use hover-only reveals** for any control (§11).
8. **Do not add scroll-triggered entrance animations.**
9. **Do not use museum vocabulary or skeuomorphic label styling** in the UI (§1).
10. **Do not use Inter, Space Grotesk, emoji iconography, gradient heroes or rounded-card layouts.**
11. **Do not put anything essential below the road** — infinite scroll makes it unreachable (§11).
12. **Do not paginate the loader on offset.** Use a slot-number cursor (§8).
13. **Do not force alt text on contributors** (§11.1).
14. **Do not store or serve user-supplied SVG.** It is XML that can carry `<script>` — stored XSS from your own origin (§10.2).
15. **Do not delete suspected CSAM unilaterally.** Quarantine, preserve, report (§7.4). Making it inaccessible is not the same as destroying it.
16. **Do not open suspected CSAM to "confirm" it.** Automated quarantine, then follow §7.4.
17. **Do not use official social share widgets or SDKs.** They track every page view whether or not anyone clicks. Plain links only (§12.4).
18. **Do not commit SECRET_KEY again,** and rotate the existing one — it is already in git history.

19. Do not invent answers to §14. Ask.

16. Companion documents

- **docs/design-plan.html** — visual direction, layout architecture, component specs, the chain diagram, and the phased build order with reasoning. Open it in a browser.
- Both files live in docs/, which is currently gitignored.